

# Module 11 — Applied Lab: Instrument the M10 Stack + RAG Smoke Evaluator

**Due:** Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

The Module 10 stack ships. The Module 11 Lab makes it tell you whether it is working. You will add three middleware layers (request-id, structured logging, metrics) to the M10 backend, declare three Prometheus metrics (counter, histogram, gauge), mount `/metrics`, and write a **3-question** smoke evaluator for `/rag/answer` that proves the grounding-check logic Thursday's Integration will scale up. The Lab is *instrumentation + a smoke evaluator*; the Integration is the *full evaluation harness*.

The instrumentation patterns are exactly what you practiced in last night's drill, applied to a larger codebase. The new work this week is the smoke evaluator — three RAG questions, three `httpx.post` calls, three pass/fail grounding checks, exit code 0 iff all three ground correctly.

## Setup

Accept the Lab template repo at <https://github.com/LevelUp-Applied-AI/ml1-ll1> — click **"Use this template"** → **"Create a new repository"**. Clone your new fork:

```
git clone https://github.com/<your-username>/ml1-ll1.git
cd ml1-ll1
git checkout -b lab-11-service-monitoring
```

Use **Python 3.11** for this template (the pinned `pydantic==2.6.0` does not build on 3.13). If your default Python is newer, create a 3.11 venv: `python3.11 -m venv .venv && source .venv/bin/activate`.

Install dependencies (cached from the drill — should be fast):

```
pip install -r requirements.txt
pip install https://github.com/explosion/spacy-models/releases/download/en_core_w
```

The spaCy model install above is only required if you plan to run `uvicorn` directly on your host. The Docker image bakes the same pinned wheel in, so `docker compose up -d` does not need it. Without the model, `/extract` silently returns a stub answer. We install the wheel by URL rather than `python -m spacy download` because the download command has been intermittently returning 404 on the resolver's GitHub URL; the pinned wheel is the stable path.

The `prometheus-client` package is already pinned in this template's `requirements.txt`, so the `pip install -r requirements.txt` step above installs it. See the [reading §1](#) for background on the package itself.

Copy the example environment file and fill in the Neo4j password and Weaviate URL (the values you used in Module 10):

```
cp .env.example .env
```

Bring up the M10 stack:

```
docker compose up -d
```

Verify the three runtime services (`neo4j`, `weaviate`, `api`) are healthy. The compose file also declares a fourth service, `web`, but it is gated behind a `frontend` Compose profile so the default `docker compose up -d` skips it (the M11 Lab does not ship Next.js source — the published "four-service stack" framing refers to the M10 topology that includes the frontend, not the Lab's runtime subset):

```
curl http://localhost:8000/readyz
```

A 200 means `api` is up and connected to Neo4j + Weaviate; if you see anything else, give the stack 60 seconds for cold starts and try again before debugging.

Seed Neo4j and Weaviate (idempotent):

```
bash seed_neo4j.sh
bash seed_weaviate.sh
```

Both seed scripts load `.env` automatically, so a learner-set `NEO4J_PASSWORD` in `.env` matches the password Docker Compose used to start the neo4j container. On **Windows Git Bash**, `seed_weaviate.sh` sets `MSYS_NO_PATHCONV=1` internally so container paths passed to `docker.exe` are not converted to Windows paths — no additional flags needed at the shell.

## What ships in the starter

```
ml1-ll1/
├── api/
│   ├── main.py          vendored M10 backend; three middleware-wire TODOs +
│   ├── models.py        vendored Pydantic models
│   ├── rag.py           vendored RAG composer
│   ├── kg.py            vendored KG mapper wrapper
│   ├── ner.py           vendored NER wrapper
│   ├── observability.py TODO file: middlewares + metric declarations
│   └── Dockerfile        vendored
├── web/                 vendored Next.js client (unchanged; not graded this
├── eval_rag_smoke.py    TODO: 3-question smoke evaluator
├── data/
│   └── rag_smoke.json    3 RAG smoke questions (question_id, question, k)
├── tests/
│   ├── test_observability.py learner-written-test placeholder
│   ├── test_metrics_endpoint.py autograder
│   ├── test_middlewares.py autograder
│   ├── test_smoke_evaluator.py autograder
│   └── test_learner_test_complete.py autograder (AST meta-check on test_observ
├── conf/test.py
├── docker-compose.yml   M10 stack
├── seed_neo4j.sh        vendored M10 seed
├── seed_weaviate.sh     vendored M10 seed
├── requirements.txt     M10 deps + prometheus-client
└── README.md
```

## Tasks

### Task 1 — Bring up the M10 stack against this starter

You did this above. Confirm `/readyz` returns 200 against `http://localhost:8000`.

### Task 2 — Implement the three middlewares and three metrics in `api/observability.py`

The file ships only TODO markers for the metric declarations and the three middleware classes — you write each class from scratch. Implement:

**Metric declarations** at module scope:

- `requests_total` — `Counter` with labels `["path", "status"]`.
- `request_latency_seconds` — `Histogram` with labels `["path"]`, default Prometheus latency buckets.
- `inflight_requests` — `Gauge`, no labels.

**RequestIdMiddleware**. Generate `uuid.uuid4()`.hex on entry. Store in a module-level `request_id_var: ContextVar[str]`. Set response header `X-Request-ID`.

**StructuredLoggingMiddleware**. Time the request. Emit one JSON line at INFO level. The autograder asserts four keys are present and the line parses as JSON: `request_id`, `path`, `status`, `latency_ms`. Best-practice fields like `ts` (timestamp) and `level` are not autograder-checked but are recommended. Use `logging.getLogger("ml1.api")` — the same logger name `main.py` already uses. The starter does not configure a JSON formatter; format the line yourself with `json.dumps(...)` and pass the resulting string to `.info(...)`.

**MetricsMiddleware**. Increment `inflight_requests` on entry; decrement on exit in `try/finally`. Time the handler. After response, increment `requests_total.labels(...).inc()` and call `request_latency_seconds.labels(...).observe(elapsed)`.

The methodology details — buckets, label sets, what counts as `path` for parameterized routes — are documented in the docstrings of the TODO functions. Follow them exactly.

### Task 3 — Wire the middlewares in `api/main.py`

In `main.py` the starter has a TODO block marked `# TODO (learner): wire the three middlewares onto app in the correct order`. Add the three `app.add_middleware(...)` calls so `request-id` is outermost and metrics is innermost — see the reading §9. Then mount `/metrics` via `make_asgi_app()` at the TODO marked `# TODO (learner): mount /metrics on app using make_asgi_app()`.

Rebuild and restart the `api` container so the new code is loaded. The compose file does not mount the `api/` source as a volume, so a plain `docker compose restart api` re-runs the same built image and never picks up your edits — use `--build`:

```
docker compose up -d --build api
```

Confirm `/metrics` is live. The `-L` flag follows the 307 redirect that Starlette issues from `/metrics` to `/metrics/`:

```
curl -L http://localhost:8000/metrics | head -30
```

You should see `# HELP` and `# TYPE` lines for the three metric families.

### Task 4 — Implement the RAG smoke evaluator in `eval_rag_smoke.py`

The starter has the function signatures. Implement:

- Load the 3 questions from `data/rag_smoke.json`.
- For each question, POST to `/rag/answer`. Build the URL from `API_URL` (read with `os.environ.get("API_URL", "http://localhost:8000")`) so the harness works inside Docker and against a remote stack — for example, `httpx.post(f"{API_URL}/rag/answer", json={"question": question["question"], "k": question.get("k", 4)}, timeout=60.0)`. The generous timeout matters — `/rag/answer` cold-cache can take ~10 s and the default 5 s timeout will flake.
- For each response, read the candidate set directly from the response body's `retrieved` field (`candidate_ids = {chunk["chunk_id"] for chunk in response_body["retrieved"]}`). Do not re-issue a separate retrieval call — the candidate set is already on the `/rag/answer` payload.
- Call `score_grounding(response_body, candidate_ids)`. Grounding is exactly two conditions: (a) `len(citations) >= 1` AND (b) every cited `chunk_id` is in `candidate_ids`. The Lab smoke evaluator does not apply decline-exclusion — the three smoke questions are all answerable against the seeded fixtures, so a decline at the Lab tier is itself a defect that the smoke evaluator should expose. Decline-exclusion is an Integration-tier concern.
- Print PASS or FAIL per question.
- Exit code 0 iff all three are grounded; non-zero otherwise.

The reading §14 states the full three-condition grounding methodology; the Lab applies only the first two conditions (decline-exclusion is the Integration's concern, not the Lab's). The `eval_rag_smoke.py` docstring as shipped states this Lab subset and is the canonical text for the Lab's grounding logic — leave the docstring as shipped. Do not copy the full reading §14 paragraph in over it; that would re-introduce decline-exclusion language that contradicts the Lab's design.

Run it against the live stack:

```
python eval_rag_smoke.py
```

Expect 3 PASS lines and exit code 0 on a healthy stack.

### Task 5 — Write your tests in `tests/test_observability.py`

The starter has `pytest.fail("Not implemented")` placeholders. Write at least **three** test functions, each with at least one `assert`. Required coverage:

- After one request, the `X-Request-ID` response header is set and non-empty.
- After one request, the `requests_total` counter for that (path, status) has incremented.
- After one request, the structured log captured during the request contains the same `request_id` that appears in the response header.

Run them:

```
pytest tests/test_observability.py -v
```

All three must pass before you push.

### Task 6 — Update `README.md`

Add a one-paragraph section titled "Observability" describing the three metric families you declared, the bucket choice for the latency histogram, and one sentence about how to read `/metrics`.

### Task 7 — Tear down

```
docker compose down -v
```

## Submission

Commit, push, open a PR within your fork. Your PR description must include:

- Confirmation that `docker compose up -d` brings up the stack and `/readyz` returns 200.
- Confirmation that `python eval_rag_smoke.py` exits 0.
- A short paragraph (~100 words) describing one design decision you made — for example, your bucket choice rationale, your label set for `requests_total`, or how you ordered the middlewares and why.
- Paste your PR URL into TalentLMS → Module 11 → Lab 11 to submit this assignment.

## Common pitfalls

- Metrics middleware outermost**. Latency includes the other middlewares; the autograder enforces innermost.
- Histogram declared but never observed**. The histogram appears in `/metrics` (with all bucket counts at 0) but no `_bucket` increments — `.observe()` was never called.
- `requests_total.inc()` without `.labels(...)`. Raises because labels are declared on the parent.
- `X-Request-ID` set on `request.headers`. Set it on `response.headers`. Requests are read-only after middleware receives them.
- `/metrics` 404. Mount path wrong, or `make_asgi_app` imported from the wrong module.
- Smoke evaluator exit code 0 even when answers are wrong**. The grounding check is missing or the candidate-set re-issue is calling a stale endpoint.
- `httpx` default timeout. `/rag/answer` can exceed 5 s. Pass `timeout=60.0`.

## Challenge Extensions

Optional. Extend the base Lab with one or more of the tiers below. Each tier introduces a substantive concept not in the base. No starter files for tiers — design the implementation from the description.

### Tier 1 — Custom histogram buckets for the RAG endpoint

The base Lab uses the default Prometheus latency buckets across all paths. The `/rag/answer` endpoint's actual latency distribution may sit far enough away from the default boundaries that p95 estimation is coarse. Tier 1: (i) author a small `httpx`-based collection script that fires 50 sequential `POST /rag/answer` calls against the seeded Weaviate and records the per-call latency; (ii) compute the empirical p50, p95, p99 from the raw observations; (iii) propose a custom bucket sequence for a new Histogram registration named `rag_request_latency_seconds` (label-value-specific bucketing is not directly supported by `prometheus_client` — declare a second histogram or argue why the global buckets are still appropriate); (iv) re-collect 50 observations after the change and produce `tier-1=` `bucket-rationale.md` comparing p95 resolution pre and post. Substantive concept: bucket design as a function of the observed distribution; the cost of cardinality (a second histogram doubles series count).

### Tier 2 — Exemplar tracker

Extend the metrics middleware to capture *one example trace per histogram bucket per (path, status)*. When a request's latency falls into a bucket whose exemplar slot is empty, store `(request_id, timestamp, response_size_bytes)`. When the slot is occupied, replace probabilistically using reservoir sampling so the exemplar is uniformly sampled across the bucket's observation history. Expose the exemplar table at a new `GET /debug/exemplars` endpoint gated by a `DEBUG=1` environment variable returning JSON. Write two pytest tests verifying the exemplar slots populate after sufficient traffic and that the reservoir behavior is correct. Substantive concept: exemplar trackers — the OpenMetrics 2.0 feature that links aggregate metrics back to specific request events — and reservoir sampling. Linking aggregates to per-event detail is a production observability pattern.

### Tier 3 — Cardinality-safe dynamic counter registry

Design and implement a `SafeCounter` abstraction that wraps `prometheus_client.Counter` and enforces a cardinality budget per metric. When `.labels(...).inc()` is called with a label combination that would exceed the budget (default 100 distinct combinations), route the call into a single overflow bucket (`label="__overflow__"`) rather than registering the new timeseries. Implement: (a) the wrapper class with an LRU-like budget enforcer; (b) per-metric configurable budget; (c) a debug endpoint `GET /debug/cardinality` returning per-metric usage; (d) integrate into the Lab's `requests_total` so the autograder smoke test exercises it; (e) `tier-3=design.md` discussing why this matters in production (the Prometheus server-side OOM that follows when an unbounded label ships) and the trade-offs (the overflow bucket loses fidelity). Substantive concept: library design for production safety — the abstraction a senior engineer ships once to protect their organization from a class of incidents.

